

Electronic Health Record: Pharmacy Organization Module

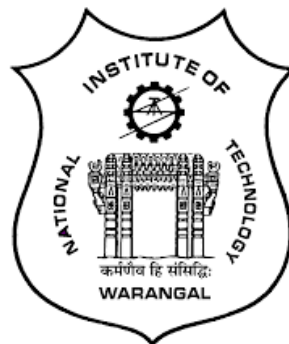
Master of Technology

COMPUTER SCIENCE AND INFORMATION SECURITY

ANKIT CHAUDHARI (25CSM2R03)

MOHIT TOMAR (25CSM2R13)

RONITSINGH RAWAT (25CSM2S06)



Under the guidance of

PROF. ERUKALA SURESH BABU

ASSOCIATE PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

April, 2026

Contents

Abstract	1
1 Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Proposed Solution	4
1.4 Objectives	5
1.5 Organization of the Report	5
2 Related Work	6
2.1 Blockchain in Healthcare	6
2.2 Hyperledger Fabric for Enterprise Applications	7
2.3 Off-Chain Storage Solutions	7
3 System Architecture	9
3.1 High-Level Architecture	9
3.2 Multi-Node Deployment Topology	10
3.3 Docker Swarm and Overlay Networking	13
3.4 Hyperledger Fabric Network Configuration	13
3.4.1 Organization and Channel	13
3.4.2 Consensus Mechanism	13
3.4.3 Certificate Authority (CA)	14
3.4.4 Endorsement Policy	14
3.5 Transaction Lifecycle	14
3.5.1 Read Transaction (Query)	14
3.5.2 Write Transaction (Invoke)	14

3.6	Gossip Protocol and Data Synchronization	15
3.7	Identity and Access Control	15
3.8	Hybrid Storage Strategy	16
4	Individual Contributions	17
4.1	Ankit Chaudhari (25CSM2R03): Infrastructure Architect & Swarm Master	17
4.1.1	Role Overview	17
4.1.2	Containers Hosted on PC1	17
4.1.3	Docker Swarm Orchestration	17
4.1.4	Deployment Automation (<code>deploy.sh</code>)	18
4.1.5	Cryptographic Material Distribution	19
4.1.6	Manager Portal	20
4.2	Mohit Tomar (25CSM2R13): Smart Contract Lead & Clinical Portals	25
4.2.1	Role Overview	25
4.2.2	Containers Hosted on PC2	25
4.2.3	Smart Contract Architecture (Chaincode)	25
4.2.4	Identity Resolution Algorithm	25
4.2.5	CouchDB Rich Queries	27
4.2.6	Patient Portal	27
4.2.7	Pharmacist / Billing Portal	29
4.3	Ronit singh Rawat (25CSM2S06): API Gateway Architect & Inventory Lead	31
4.3.1	Role Overview	31
4.3.2	Containers Hosted on PC3	31
4.3.3	API Gateway Architecture	31
4.3.4	Smart Routing Algorithm	32
4.3.5	Wallet and Identity Management	32
4.3.6	High Availability and Peer Redundancy	33
4.3.7	Inventory Application	34
4.4	Verification and Results	36
4.4.1	Ledger Synchronization Verification	36
4.4.2	CouchDB State Verification	36

4.4.3	System Specifications	37
5	Conclusion and Future Scope	38
5.1	Conclusion	38
5.2	Future Scope	39

List of Figures

3.1	Logical Component and Swarm Communication Architecture for the EHR System	10
3.2	Comprehensive Container Map & Topology Deployment	12
3.3	Transaction Flow Diagram: Read Operation (Evaluate)	15
3.4	Transaction Flow Diagram: Write Operation (Submit)	16
4.1	Manager Portal: Dashboard and Component Overview	20
4.2	Manager Portal: Employee Enrollment and Role Assignment	21
4.3	Manager Portal: Swarm Network Health Monitoring	22
4.4	Manager Portal: Global Blockchain Audit Trail	23
4.5	Manager Portal: Read-Only Global Inventory Oversight	24
4.6	Patient Portal: Self-Sovereign Health Record Access (Part 1)	28
4.7	Patient Portal: Self-Sovereign Health Record Access (Part 2)	28
4.8	Pharmacist Portal: Prescription Fulfillment and Billing Interface (Part 1)	30
4.9	Pharmacist Portal: Prescription Fulfillment and Billing Interface (Part 2)	30
4.10	State Mirroring: CouchDB2 Interface on PC3	33
4.11	Inventory Portal: Adding New Medicine Stock to the Ledger	35
4.12	Inventory Portal: Medicine Stock Management and Deduction View	35

List of Tables

2.1	Comparison of Blockchain Platforms for Healthcare	7
3.1	Node Deployment Topology	11
4.1	PC1 Container Inventory (Ankit Chaudhari)	18
4.2	PC2 Container Inventory (Mohit Tomar)	25
4.3	Chaincode Asset Types	26
4.4	PC3 Container Inventory (Ronitsingh Rawat)	31
4.5	Development and Deployment Environment	37

Abstract

Modern healthcare systems suffer from centralized and fragmented data management, leading to critical challenges in data privacy, interoperability, and patient empowerment. The **Electronic Health Record (EHR) System** is an enterprise-grade, decentralized platform built upon **Hyperledger Fabric v2.5** and orchestrated across a multi-node **Docker Swarm** cluster, specifically tailored for pharmacy organization workflows. The system employs a permissioned blockchain to guarantee data immutability, cryptographic identity management via X.509 certificates, and granular Attribute-Based Access Control (ABAC) to regulate data visibility across Managers, Pharmacists, and Patients.

The platform is deployed across three geographically distributed physical machines interconnected through a **Tailscale VPN** mesh, forming a secure overlay network. The architecture features a Raft-based Ordering Service for consensus, CouchDB-backed rich queries for high-performance state lookups, and IPFS integration for off-chain storage of heavy medical documents. A Node.js API Gateway bridges the React-based frontend portals to the blockchain network using the official **fabric-network** SDK, translating REST requests into cryptographically signed gRPC transactions.

The system comprises four role-based portals: a **Manager Portal** for governance and identity provisioning, a **Patient Portal** for self-sovereign health record access, a **Pharmacist Portal** for prescription fulfillment with privacy-preserving data filtering, and an **Inventory Portal** for immutable medicine supply chain tracking. The project demonstrates a fully functional, production-ready decentralized healthcare infrastructure with automated one-click deployment, fault-tolerant peer redundancy, and real-time ledger synchronization via the Gossip protocol.

Keywords: Hyperledger Fabric, Docker Swarm, Electronic Health Records, Blockchain,

IPFS, ABAC, CouchDB, Decentralized Application.

Chapter 1

Introduction

1.1 Background

Healthcare systems worldwide rely heavily on centralized databases for storing and managing Electronic Health Records (EHR). While centralized systems offer simplicity in deployment, they introduce critical vulnerabilities including single points of failure, data tampering risks, and fragmented patient data across institutions. Patients have limited control over who accesses their sensitive medical information, and interoperability between hospitals remains a persistent challenge.

1.2 Problem Statement

The current landscape of healthcare data management suffers from the following fundamental challenges:

1. **Data Fragmentation:** Patient records are scattered across multiple hospital databases with no unified access mechanism. Transferring records between institutions requires manual, error-prone processes.
2. **Privacy and Ownership:** Patients lack sovereign control over their own medical data. Unauthorized access to sensitive diagnoses is difficult to prevent and audit in traditional relational database systems.
3. **Data Integrity:** Centralized databases are susceptible to unauthorized modification. Medical records can be altered or deleted without an immutable

audit trail, compromising trust between healthcare providers.

4. **Supply Chain Opacity:** Pharmaceutical inventory tracking in hospitals lacks transparency, enabling potential fraud through ghost prescriptions or unaccounted medicine dispensation.
5. **Scalability of Trust:** As healthcare networks grow across geographic boundaries, establishing mutual trust between disparate entities using traditional certificate-based or password-based authentication becomes increasingly complex.

1.3 Proposed Solution

The Electronic Health Record (EHR) system addresses these challenges by implementing a **permissioned blockchain-based EHR platform using Hyperledger Fabric v2.5**. Unlike public blockchains (e.g., Ethereum), Hyperledger Fabric provides enterprise-grade features including:

- **Permissioned Access:** Only authorized entities with valid X.509 certificates can participate in the network.
- **Channel-based Privacy:** Data is isolated within channels, preventing unauthorized cross-organizational data leakage.
- **Pluggable Consensus:** The Raft consensus mechanism provides crash fault tolerance without the computational overhead of Proof-of-Work.
- **Rich Queries:** CouchDB integration enables complex JSON-based queries on the world state for efficient data retrieval.

The system is deployed across a **three-node Docker Swarm cluster** connected via a **Tailscale VPN mesh**, demonstrating true decentralization across geographically separated physical machines. An **IPFS** layer provides off-chain storage for heavy medical documents, keeping the blockchain lightweight while maintaining content-addressed integrity.

1.4 Objectives

The primary objectives of the Electronic Health Record (EHR) project are:

1. Design and deploy a multi-node Hyperledger Fabric network across physically distributed machines using Docker Swarm orchestration.
2. Develop smart contracts (chaincode) that enforce immutable rules for patient registration, prescription management, access control, and inventory tracking.
3. Build role-based frontend portals (Manager, Patient, Pharmacist, Inventory) with cryptographic identity-driven access control.
4. Integrate IPFS for decentralized off-chain storage of medical documents with on-chain hash anchoring.
5. Automate the entire deployment lifecycle through shell scripts for reproducible, one-click network bootstrapping.

1.5 Organization of the Report

The remainder of this report is organized as follows:

Chapter 2: Reviews related work in blockchain-based healthcare systems.

Chapter 3: Presents the overall system architecture, including network topology, consensus, and communication infrastructure.

Chapter 4: Details the implementation methodology and individual contributions of each team member, including pseudo code, walkthrough screenshots, and technical deep-dives.

Chapter 5: Concludes the report with a summary of achievements and future research directions.

Chapter 2

Related Work

The application of blockchain technology to healthcare data management has received significant attention in recent literature. This chapter reviews existing approaches, highlights their limitations, and positions the proposed EHR system within the current research landscape.

2.1 Blockchain in Healthcare

Azaria et al. (2016) proposed MedRec, one of the earliest blockchain-based EHR systems built on Ethereum. MedRec used smart contracts to manage access permissions and provided patients with an immutable log of their medical history. However, being built on a public blockchain, MedRec suffered from high transaction costs (gas fees), slow throughput, and lack of data privacy, as all transactions were visible on the public ledger.

Xia et al. (2017) introduced MeDShare, a system for sharing medical data among cloud service providers using a blockchain-based access control mechanism. While MeDShare addressed data provenance and auditability, it relied on a centralized cloud backend for actual data storage, partially negating the decentralization benefits of blockchain.

Dubovitskaya et al. (2020) developed a framework for managing and sharing Electronic Medical Records (EMRs) using Hyperledger Fabric. Their work demonstrated the advantages of permissioned blockchains for healthcare, including improved throughput and privacy. However, their deployment was limited to a single-

machine test environment and did not address multi-node distributed deployment challenges.

2.2 Hyperledger Fabric for Enterprise Applications

Hyperledger Fabric has emerged as the preferred blockchain platform for enterprise healthcare applications due to its permissioned architecture, modular design, and support for private data collections. Key differentiators include:

Table 2.1: Comparison of Blockchain Platforms for Healthcare

Feature	Ethereum	Bitcoin	Hyperledger Fabric	Our EHR System
Network Type	Public	Public	Permissioned	Permissioned
Consensus	PoS	PoW	Raft/PBFT	Raft
Smart Contracts	Solidity	Script	Go/Node.js	Node.js
Throughput	30 TPS	7 TPS	3000 TPS	3000 TPS
Data Privacy	Low	Low	High (Channels)	High (ABAC)
Off-chain Storage	External	None	External	IPFS

2.3 Off-Chain Storage Solutions

Storing large medical documents (X-ray images, lab reports, PDFs) directly on the blockchain is impractical due to block size limitations and storage costs. Several approaches have been proposed:

- **IPFS (InterPlanetary File System):** A content-addressed, peer-to-peer storage network where files are identified by their cryptographic hash. Changing a file changes its hash, providing inherent tamper detection. The proposed EHR system adopts this approach.

- **Cloud Storage with Hash Anchoring:** Files are stored on centralized cloud providers (AWS S3, Azure Blob) with only the hash recorded on-chain. This introduces a centralized dependency.
- **Private Data Collections:** Hyperledger Fabric supports sideDB for storing private data that is shared only between authorized organizations. While useful for inter-organizational privacy, it does not solve the large-file storage problem.

Chapter 3

System Architecture

This chapter presents the overall system architecture of the Electronic Health Record (EHR) System, covering the network topology, distributed infrastructure, consensus mechanism, communication protocols, and data flow patterns.

3.1 High-Level Architecture

The EHR System follows a layered architecture consisting of four tiers:

1. **Presentation Layer:** React.js-based frontend portals (Manager, Patient, Pharmacist, Inventory).
2. **Application Layer:** Node.js API Gateway bridging REST/HTTP requests to blockchain gRPC calls via the Fabric SDK.
3. **Blockchain Layer:** Hyperledger Fabric v2.5 network with Raft Ordering, Peers, CouchDB state databases, and Node.js Chaincode.
4. **Storage Layer:** IPFS for off-chain document storage with on-chain hash anchoring.

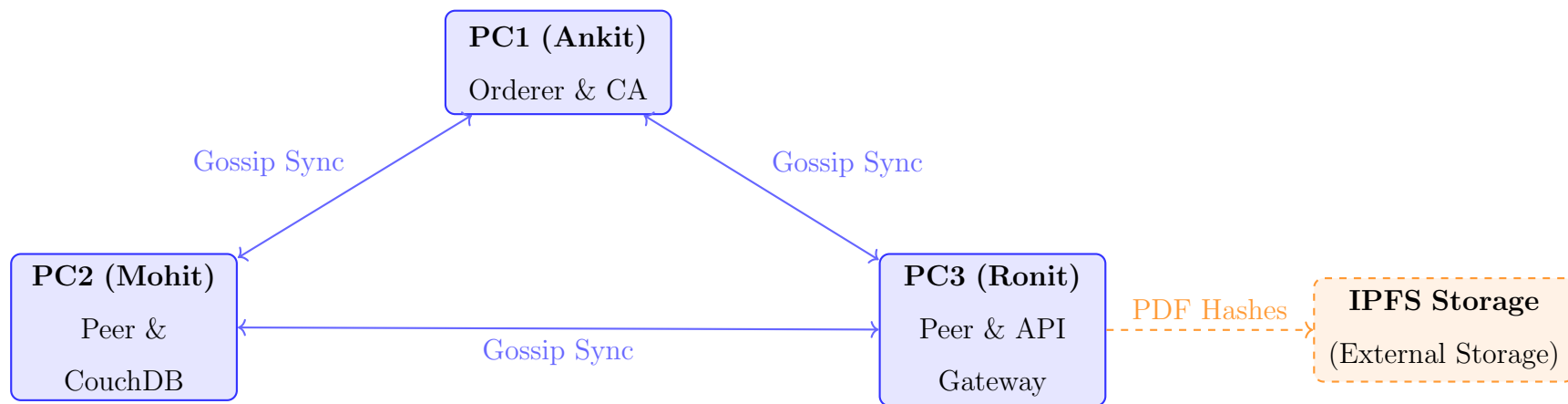


Figure 3.1: Logical Component and Swarm Communication Architecture for the EHR System

10

3.2 Multi-Node Deployment Topology

The system is deployed across three physical machines (PCs) interconnected via a **Tailscale VPN mesh**. Docker Swarm orchestrates container placement across the cluster.

Table 3.1: Node Deployment Topology

Node	Role	Operator	Key Components
PC1 (192.168.1.10)	Swarm Manager	Ankit	Orderer, CA, Peer0, CouchDB0, Manager UI
PC2 (192.168.1.11)	Swarm Worker	Mohit	Peer1, CouchDB1, Patient UI, Pharmacist UI
PC3 (192.168.1.12)	Swarm Worker	Ronit	Peer2, CouchDB2, API Gateway, Inventory UI

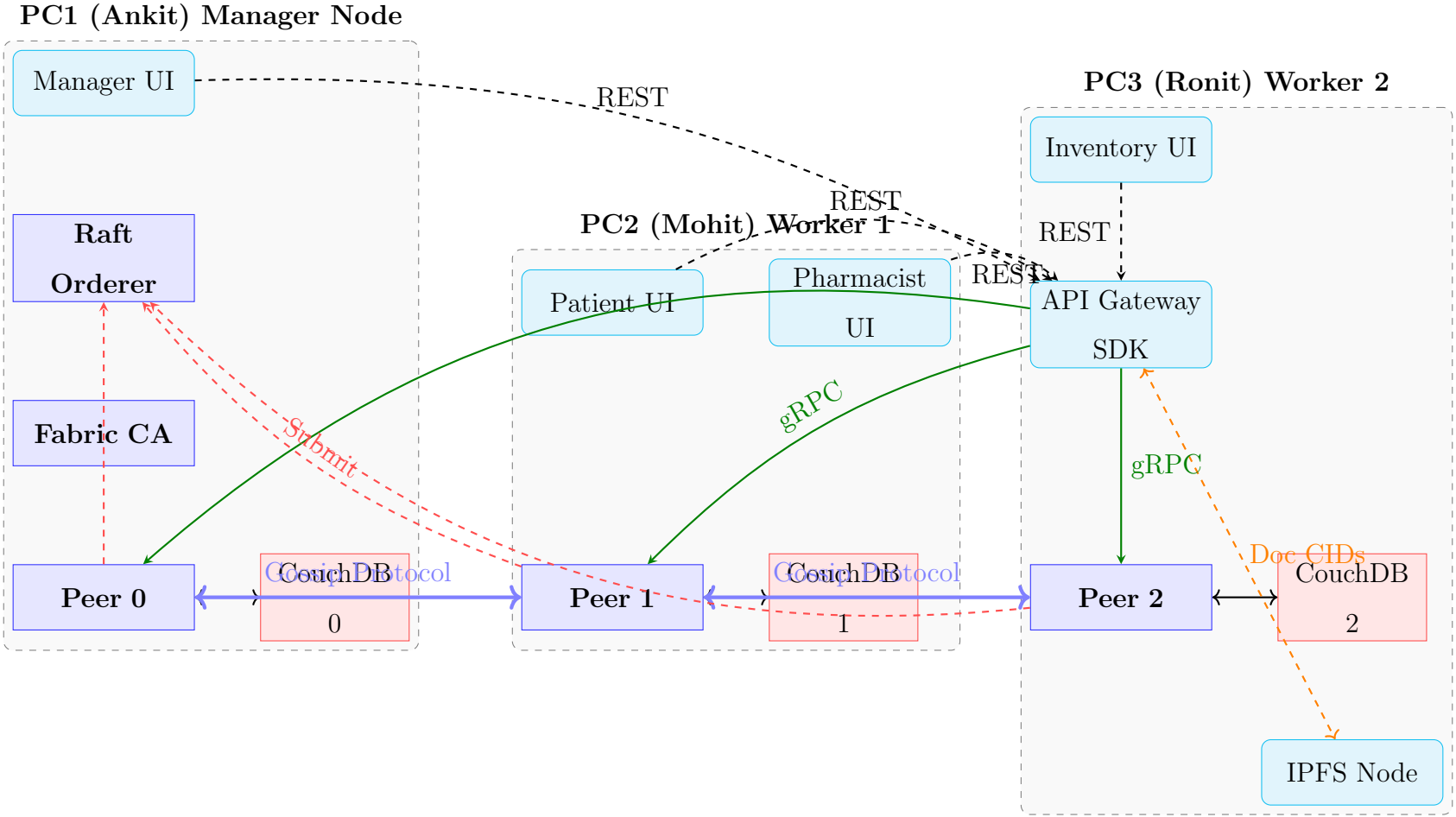


Figure 3.2: Comprehensive Container Map & Topology Deployment

3.3 Docker Swarm and Overlay Networking

Docker Swarm provides the orchestration layer that binds the three geographically separated machines into a unified logical cluster.

- **Swarm Initialization:** PC1 acts as the Swarm Manager node. PC2 and PC3 join as Worker nodes using secure join tokens.
- **Overlay Network:** A Docker overlay network (`fabric-swarm-net`) is created with the `--attachable` flag, enabling cross-node container communication through encrypted VXLAN tunnels.
- **Tailscale VPN:** Since the nodes are not on the same physical LAN, Tailscale creates a WireGuard-based mesh VPN. This provides each node with a stable, private IP address (100.x.x.x) that Docker Swarm uses for inter-node communication.
- **Service Discovery:** Docker Swarm handles DNS-based service discovery. Containers reference each other by service name (e.g., `peer1.org1.example.com`) rather than IP addresses.

3.4 Hyperledger Fabric Network Configuration

3.4.1 Organization and Channel

The network operates as a single organization (`Org1MSP`) with all three peers joined to a single channel (`mychannel`). Peer0 is configured as the **Anchor Peer** for Org1 to facilitate cross-node gossip-based peer discovery.

3.4.2 Consensus Mechanism

The EHR system uses the **Raft consensus protocol** for the Ordering Service. Raft provides:

- Crash Fault Tolerance (CFT) without the computational overhead of Byzantine Fault Tolerance (BFT).

- Leader-based ordering with automatic leader election upon failure.
- Strong consistency guarantees suitable for single-organization deployments.

3.4.3 Certificate Authority (CA)

The Fabric CA running on PC1 is the root of trust for the entire network. It generates X.509 certificates for:

- Peer node identities (for network participation)
- Admin identities (for channel and chaincode management)
- User identities (for application-level authentication)

3.4.4 Endorsement Policy

The chaincode uses the default lifecycle endorsement policy: **ANY member of Org1MSP**. This means a transaction is considered valid if endorsed by at least one peer, enabling efficient load-balanced transaction processing across the Swarm.

3.5 Transaction Lifecycle

Every transaction in the EHR System follows the Hyperledger Fabric **Execute-Order-Validate** paradigm:

3.5.1 Read Transaction (Query)

A Read transaction queries the current ledger state without modifying it. It bypasses the Orderer entirely, resulting in high throughput and low latency.

3.5.2 Write Transaction (Invoke)

A Write transaction alters the permanent blockchain state. It follows the three-step Execute-Order-Validate flow to ensure consensus and immutability across the Swarm.

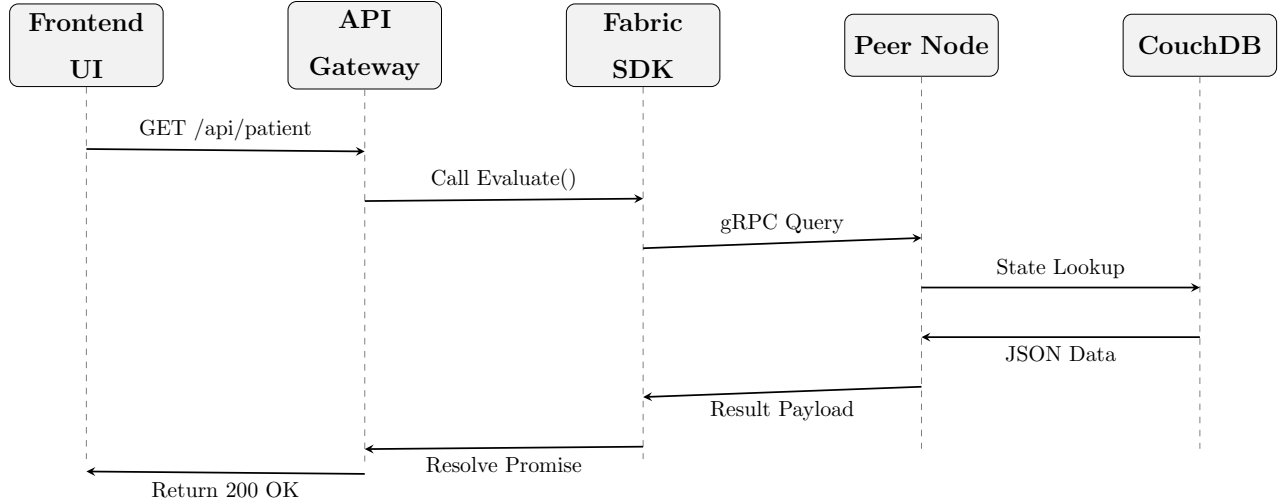


Figure 3.3: Transaction Flow Diagram: Read Operation (Evaluate)

3.6 Gossip Protocol and Data Synchronization

The Gossip protocol provides three critical functions:

- **Peer Discovery:** Peers periodically exchange heartbeat messages to detect online/offline status of other peers across the Swarm.
- **Block Dissemination:** Instead of the Orderer sending blocks to every peer individually, the Orderer delivers blocks to a leader peer, which gossips the data to remaining peers.
- **State Reconciliation:** If a peer goes offline and misses blocks, the Gossip protocol automatically detects the height mismatch and transfers the missing blocks from neighbouring peers.

3.7 Identity and Access Control

The EHR System implements a multi-layered security model:

1. **Authentication (Peer Level):** Every transaction proposal includes the sender's X.509 certificate. The Peer cryptographically verifies the certificate's validity and issuing CA before processing.
2. **Authorization (Chaincode Level):** The Smart Contract implements **Attribute-Based Access Control (ABAC)**. It reads the sender's role (Manager, Billing,

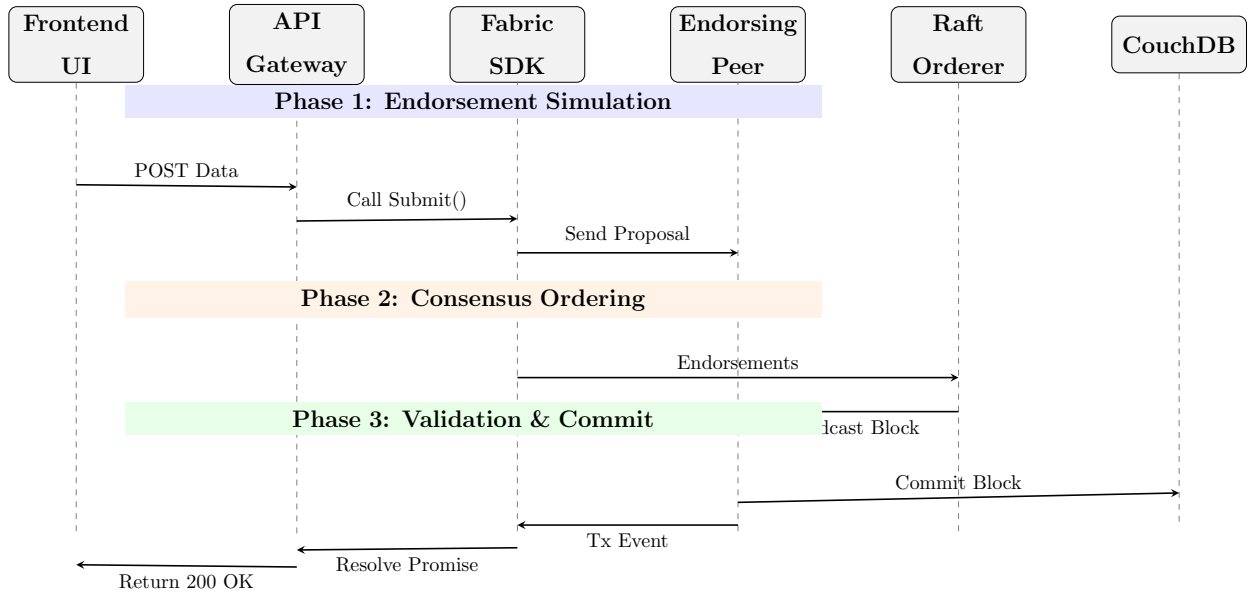


Figure 3.4: Transaction Flow Diagram: Write Operation (Submit)

Inventory, Patient) from the certificate or ledger-based employee registry and enforces function-level permissions.

3.8 Hybrid Storage Strategy

- **On-Chain (CouchDB):** Lightweight metadata such as Patient IDs, Employee records, Inventory levels, and Bill references are stored directly on the blockchain ledger for high-frequency querying.
- **Off-Chain (IPFS):** Heavy medical documents (prescriptions, scans, lab reports) are uploaded to IPFS. The API Gateway retrieves the content-addressed **CID (Content Identifier)** hash and records only this hash on the blockchain. This keeps the blockchain lightweight while ensuring document integrity through cryptographic hash verification.

Chapter 4

Individual Contributions

This chapter presents the complete implementation details of the Electronic Health Record (EHR) System, organized by the individual contributions of each team member. Each section describes the components built, the containers hosted, the technical methodology, relevant pseudo code, and application walkthrough.

4.1 Ankit Chaudhari (25CSM2R03): Infrastructure Architect & Swarm Master

4.1.1 Role Overview

Ankit is responsible for the core network infrastructure, including Docker Swarm orchestration, deployment automation, cryptographic material management, and the Manager Governance Portal. He operates PC1, the Swarm Manager node.

4.1.2 Containers Hosted on PC1

4.1.3 Docker Swarm Orchestration

The deployment leverages Docker Swarm's stack-based service definitions. Each Fabric component is defined as a separate Docker Compose stack YAML file:

- `compose/stack-ca.yaml` — Fabric CA service
- `compose/stack-orderer.yaml` — Raft Orderer service

Table 4.1: PC1 Container Inventory (Ankit Chaudhari)

Container	Image	Purpose
Fabric CA	hyperledger/fabric-ca:1.5	Root Certificate Authority for PKI
Raft Orderer	hyperledger/fabric-orderer:2.5	Transaction ordering and block creation
Peer0	hyperledger/fabric-peer:2.5	Anchor peer for Org1MSP
CouchDB0	couchdb:3.3.2	State database for Peer0
Manager Portal	Node.js / React (Vite)	Governance UI for identity provisioning

- `compose/stack-peer0.yaml` — Peer0 and CouchDB0
- `compose/stack-peer1.yaml` — Peer1 and CouchDB1 (deployed to PC2)
- `compose/stack-peer2.yaml` — Peer2 and CouchDB2 (deployed to PC3)

Swarm placement constraints ensure containers run on the correct physical node:

```

1 deploy:
2   placement:
3     constraints:
4       - node.labels.role == worker2

```

Listing 4.1: Placement Constraint Example (`stack-peer1.yaml`)

4.1.4 Deployment Automation (`deploy.sh`)

The `deploy.sh` script automates the entire network lifecycle. It is a single-command deployment tool that performs the following steps:

Algorithm 1 One-Click Network Deployment

```

1: procedure DEPLOY
2:   Step 1: Stop all existing Docker stacks and kill running application processes
3:   Step 2: Wipe old ledger data volumes on PC1, PC2, and PC3 (via SSH)
4:   Step 3: Start the IPFS daemon on PC1
5:   Step 4: Package the EHR chaincode into ehr.tar.gz
6:   Step 5: Deploy the Fabric CA and Orderer stacks on PC1
7:   Step 6: Create the channel genesis block (mychannel.block)
8:   Step 7: Sync crypto-materials and channel block to PC2 and PC3 via SCP
9:   Step 8: Deploy Peer0, Peer1, and Peer2 stacks across the Swarm
10:  Step 9: Join all three peers to mychannel
11:  Step 10: Install, approve, and commit the chaincode lifecycle
12:  Step 11: Import wallet identities for the API Gateway
13:  Step 12: Start the Node.js backend and all React frontend portals
14:  Step 13: Seed initial dummy data for demonstration
15: end procedure

```

4.1.5 Cryptographic Material Distribution

A critical challenge in multi-node Fabric deployment is ensuring all worker nodes possess the correct MSP (Membership Service Provider) credentials before their peer containers start. The `deploy.sh` script solves this via automated SCP transfers:

```

1 # Sync crypto-config, channel block, and chaincode to PC2
2 scp -r crypto-config/ rajput_mt@100.83.121.98:~/fabric-
   network-swarm/
3 scp mychannel.block    rajput_mt@100.83.121.98:~/fabric-
   network-swarm/
4 scp ehr.tar.gz         rajput_mt@100.83.121.98:~/fabric-
   network-swarm/
5
6 # Sync to PC3
7 scp -r crypto-config/ ronit@100.117.138.55:~/fabric-network-
   swarm/

```

```
8 scp mychannel.block      ronit@100.117.138.55:~/fabric-network-  
    swarm/  
9 scp ehr.tar.gz          ronit@100.117.138.55:~/fabric-network-  
    swarm/
```

Listing 4.2: Crypto-Material Sync to Worker Nodes

4.1.6 Manager Portal

The Manager Portal is a React-based governance interface that allows hospital administrators to:

Functional Procedures

1. **Manager Dashboard & Overview:** The dashboard acts as the primary landing page, giving the administrator a high-level view of system modules.

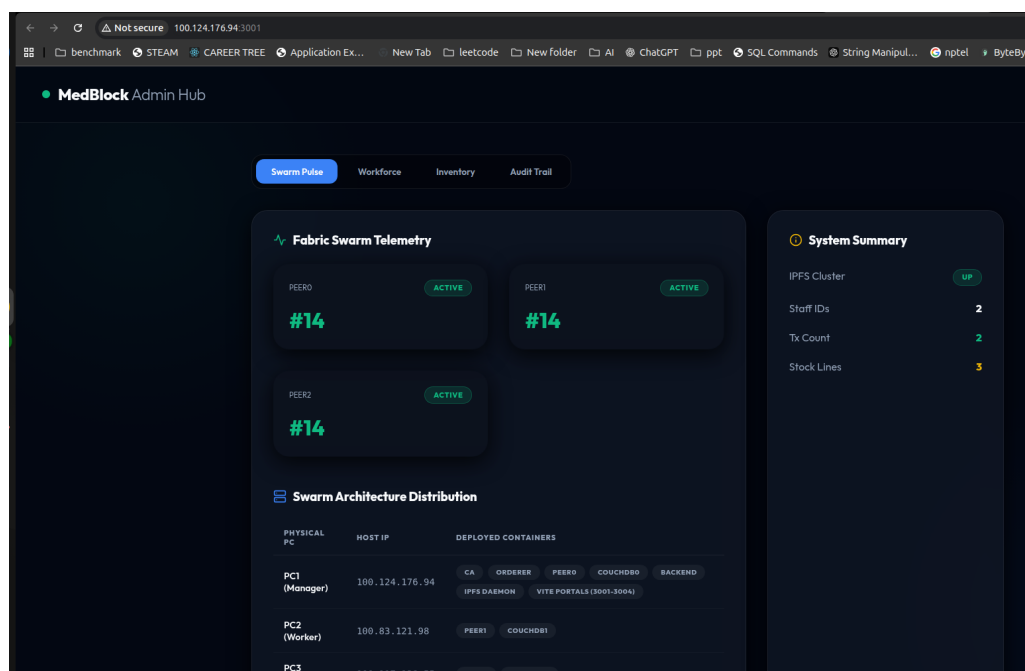


Figure 4.1: Manager Portal: Dashboard and Component Overview

2. Registering a New Employee:

1. The Hospital Admin logs into the Manager Portal using their secure administrative credentials.

2. Navigates to the “Workforce Registry” tab on the side-menu interface.
3. Enters the employee’s details (Staff ID, Full Name) and selects the specific Role (doctor, pharmacist, or inventory).
4. Upon clicking “Register”, the React frontend submits a POST request to the API Gateway.
5. The Gateway bridges to the blockchain, invoking the `registerEmployee` smart contract function.
6. The identity and role are permanently written into CouchDB under the `EMP_` namespace, and the Fabric CA provisions the employee’s cryptographic wallet.

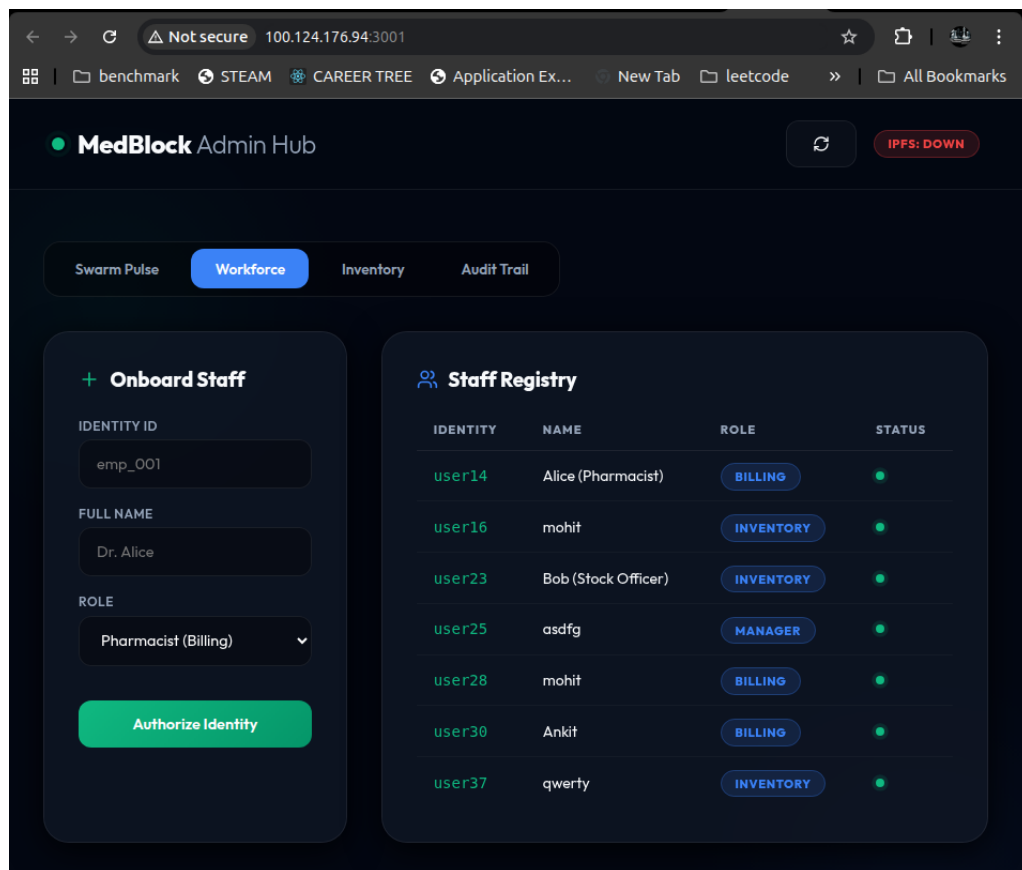


Figure 4.2: Manager Portal: Employee Enrollment and Role Assignment

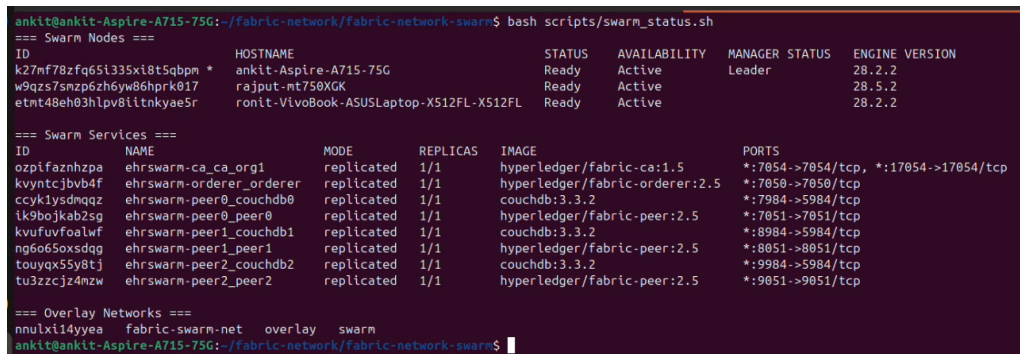
3. Registering a New Patient:

1. The Admin navigates to the “Patient Enrollment” tab.
2. Enters the patient’s demographic details (National ID, Date of Birth).

3. Submits the form, invoking the `createPatient` chaincode transaction.
4. The ledger records the patient. Crucially, the system issues the patient a deterministic mapping to an X.509 Alias certificate, creating the cryptographic foundation for their self-sovereign identity access later.

4. Network Auditing and Block Monitoring:

1. Navigating to the “Network Health” tab fetches real-time ledger height metrics via the SDK, confirming that Gossip synchronization is functioning across PC1, PC2, and PC3.
2. Navigating to the “Global Audit” tab allows the manager to pull all transparent hospital records. The chaincode executes a CouchDB Mango query across all `BILL_` documents, giving administrators total oversight of activities.



```

ankit@ankit-Aspire-A715-75G:~/fabric-network/fabric-network-swarm$ bash scripts/swarm_status.sh
=== Swarm Nodes ===

```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
k27mf78zfq65i335xi8t5qbpn *	ankit-Aspire-A715-75G	Ready	Active	Leader	28.2.2
w9qzs7smzp6zh6yw86hprk017	rajput-mt758XGK	Ready	Active		28.5.2
etnt48eh03hlpv8iitnkyae5r	ronit-VivoBook-ASUSLaptop-X512FL-X512FL	Ready	Active		28.2.2

```

=== Swarm Services ===

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
ozpifaznhzpa	ehrswarm-ca_ca_org1	replicated	1/1	hyperledger/fabric-ca:1.5	*:7054->7054/tcp, *:17054->17054/tcp
kvyntcjbbv4f	ehrswarm-orderer_orderer	replicated	1/1	hyperledger/fabric-orderer:2.5	*:7050->7050/tcp
ccyk1ysdmqz	ehrswarm-peer0_couchdb0	replicated	1/1	couchdb:3.3.2	*:7984->5984/tcp
ik9bojkab2sg	ehrswarm-peer0_peer0	replicated	1/1	hyperledger/fabric-peer:2.5	*:7051->7051/tcp
kvufuyfoalwf	ehrswarm-peer1_couchdb1	replicated	1/1	couchdb:3.3.2	*:8984->5984/tcp
ng6o65oxsdqg	ehrswarm-peer1_peer1	replicated	1/1	hyperledger/fabric-peer:2.5	*:8051->8051/tcp
touyx55y8tj	ehrswarm-peer2_couchdb2	replicated	1/1	couchdb:3.3.2	*:9984->5984/tcp
tu3zzczjz4mzw	ehrswarm-peer2_peer2	replicated	1/1	hyperledger/fabric-peer:2.5	*:9051->9051/tcp

```

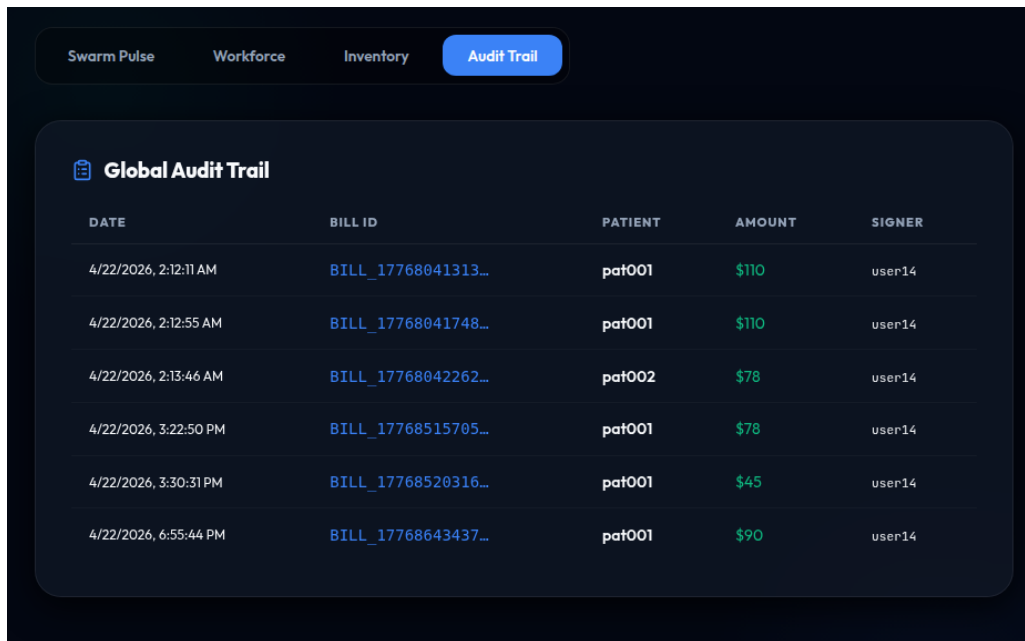
=== Overlay Networks ===
nnulxi4yyea fabric-swarm-net overlay swarm
ankit@ankit-Aspire-A715-75G:~/fabric-network/fabric-network-swarm$

```

Figure 4.3: Manager Portal: Swarm Network Health Monitoring

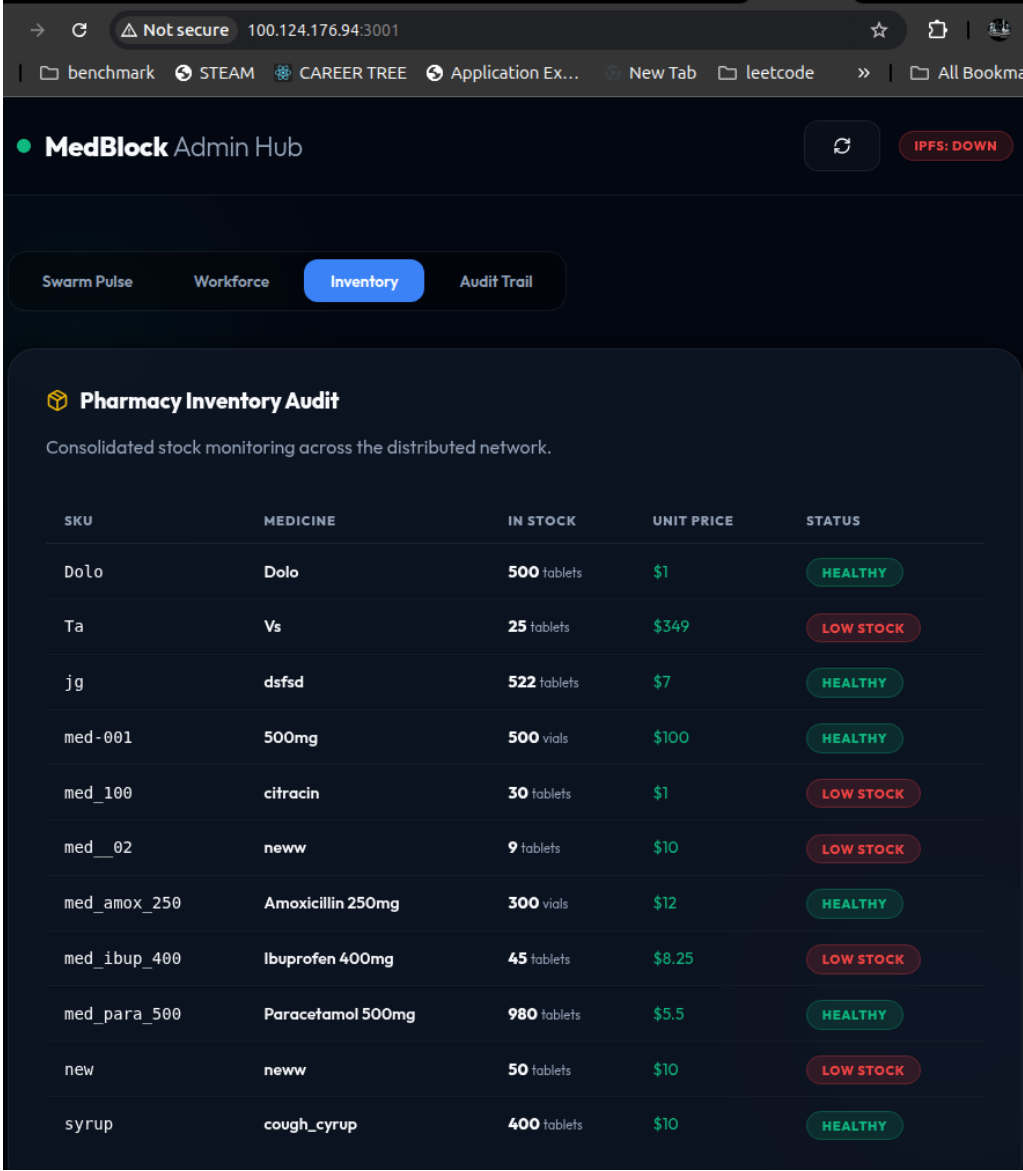
5. Hospital Inventory Oversight:

1. The manager can independently view global stock levels recorded on the ledger.
2. Accessing the “Stock Monitor” tab queries the `getAllInventory` chaincode, retrieving the immutable, real-time aggregated state of medicine quantities currently managed by the Inventory staff on PC3.



DATE	BILL ID	PATIENT	AMOUNT	SIGNER
4/22/2026, 2:12:11 AM	BILL_17768041313...	pat001	\$110	user14
4/22/2026, 2:12:55 AM	BILL_17768041748...	pat001	\$110	user14
4/22/2026, 2:13:46 AM	BILL_17768042262...	pat002	\$78	user14
4/22/2026, 3:22:50 PM	BILL_17768515705...	pat001	\$78	user14
4/22/2026, 3:30:31 PM	BILL_17768520316...	pat001	\$45	user14
4/22/2026, 6:55:44 PM	BILL_17768643437...	pat001	\$90	user14

Figure 4.4: Manager Portal: Global Blockchain Audit Trail



MedBlock Admin Hub

Swarm Pulse Workforce **Inventory** Audit Trail

Pharmacy Inventory Audit

Consolidated stock monitoring across the distributed network.

SKU	MEDICINE	IN STOCK	UNIT PRICE	STATUS
Dolo	Dolo	500 tablets	\$1	HEALTHY
Ta	Vs	25 tablets	\$349	LOW STOCK
jg	dsfsd	522 tablets	\$7	HEALTHY
med-001	500mg	500 vials	\$100	HEALTHY
med_100	citracin	30 tablets	\$1	LOW STOCK
med_02	neww	9 tablets	\$10	LOW STOCK
med_amox_250	Amoxicillin 250mg	300 vials	\$12	HEALTHY
med_ibup_400	Ibuprofen 400mg	45 tablets	\$8.25	LOW STOCK
med_para_500	Paracetamol 500mg	980 tablets	\$5.5	HEALTHY
new	neww	50 tablets	\$10	LOW STOCK
syrup	cough_cyrup	400 tablets	\$10	HEALTHY

Figure 4.5: Manager Portal: Read-Only Global Inventory Oversight

4.2 Mohit Tomar (25CSM2R13): Smart Contract Lead & Clinical Portals

4.2.1 Role Overview

Mohit is responsible for the Fabric Chaincode (Smart Contracts) that define all business logic, the CouchDB state database on PC2, and the clinical-facing frontend applications (Patient Portal and Pharmacist/Billing Portal).

4.2.2 Containers Hosted on PC2

Table 4.2: PC2 Container Inventory (Mohit Tomar)

Container	Image	Purpose
Peer1	hyperledger/fabric-peer:2.5	Endorsing peer for transaction simulation
CouchDB1	couchdb:3.3.2	State database with rich query support
Patient Portal	React (Vite)	Patient self-governance interface
Pharmacist Portal	React (Vite)	Prescription billing interface

4.2.3 Smart Contract Architecture (Chaincode)

The EHR chaincode is implemented in Node.js using the `fabric-contract-api` library. It manages four distinct asset types on the ledger:

Key File: `chaincode/ehr/index.js`

4.2.4 Identity Resolution Algorithm

A critical component of the chaincode is the `_getRole()` function, which determines the caller's role through a multi-step resolution process:

Table 4.3: Chaincode Asset Types

Asset	Key Prefix	docType	Description
Employee	EMP_	employee	Hospital staff identity records
Patient	PATIENT_	patient	Patient demographic and history
Bill	BILL_	bill	Prescriptions and billing records
Inventory	INV_	inventory	Medicine stock tracking

Algorithm 2 Identity Resolution and Role Detection

```

1: procedure GETROLE(ctx)
2:   role  $\leftarrow$  GETCERTATTRIBUTE(ctx, “role”)
3:   if role  $\neq$  null then
4:     return role ▷ Explicit CA attribute
5:   end if
6:   id  $\leftarrow$  GETCLIENTID(ctx)
7:   if id contains “admin” or “manager” then
8:     return “manager”
9:   end if
10:  alias  $\leftarrow$  EXTRACTCN(id) ▷ Extract Common Name from X.509 DN
11:  empData  $\leftarrow$  GETSTATE(“EMP_” + alias)
12:  if empData  $\neq$  null and empData.isActive then
13:    return empData.role ▷ Ledger-based registry lookup
14:  end if
15:  patientLookup  $\leftarrow$  GETSTATE(“ALIAS_” + alias + “_PATIENT”)
16:  if patientLookup  $\neq$  null then
17:    return “patient” ▷ Reverse alias patient lookup
18:  end if
19:  return “guest” ▷ Fallback: no permissions
20: end procedure

```

4.2.5 CouchDB Rich Queries

By configuring `CORE_LEDGER_STATE_STATEDATABASE=CouchDB` in the peer environment, Mohit enabled the chaincode to execute JSON-based (Mango) queries. This replaces the default LevelDB key-value lookups with indexed, attribute-based searches:

```
1 // Fetch all bills for a specific patient
2 async getPatientBills(ctx, patientId) {
3     const iterator = await ctx.stub.getStateByRange('', '');
4     const results = [];
5     while (true) {
6         const result = await iterator.next();
7         if (result.value) {
8             const record = JSON.parse(result.value.value.
9                 toString());
10            if (record.docType === 'bill' && record.patientId
11                === patientId) {
12                results.push(record);
13            }
14        }
15        if (result.done) { await iterator.close(); break; }
16    }
17    return JSON.stringify(results);
18 }
```

Listing 4.3: CouchDB Rich Query Example (Chaincode)

4.2.6 Patient Portal

The Patient Portal embodies the concept of **Self-Sovereign Identity**. When a patient logs in, the system maps their physical ID to a deterministic X.509 certificate. The chaincode then ensures they can only access records where their `ownerAlias` matches.

Key Features:

- View complete chronological medical history

- Download medical reports via IPFS CID hash retrieval
- Grant or revoke access to specific healthcare providers
- View list of entities currently authorized to access records

Key File: `app/patient/src/App.jsx`

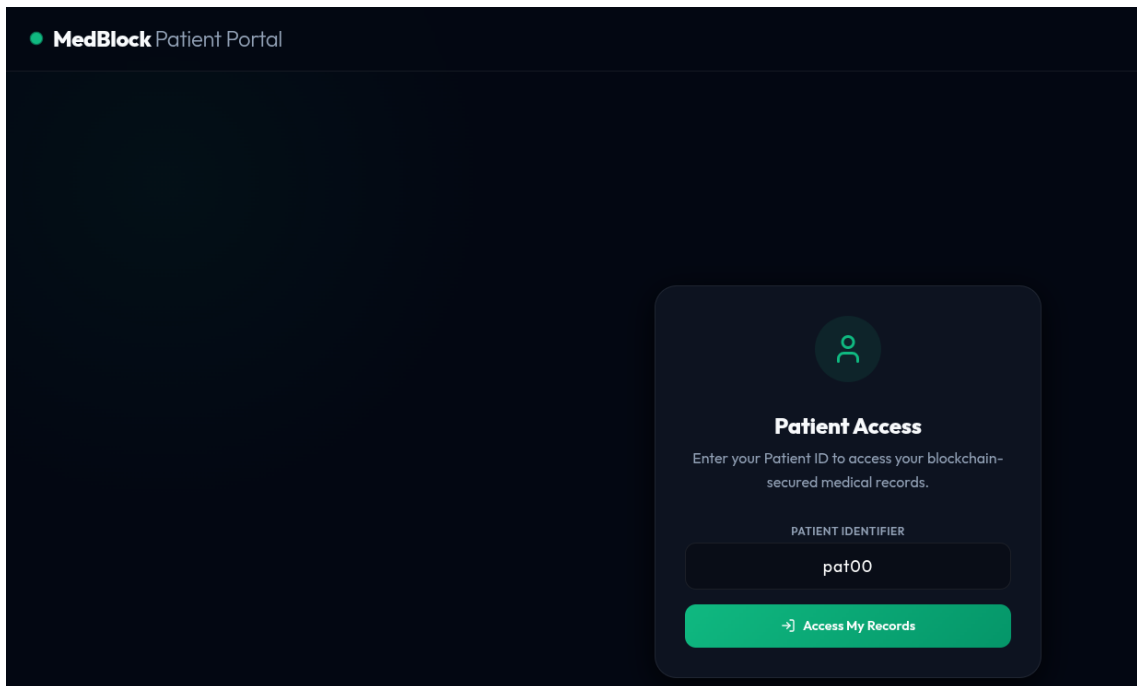


Figure 4.6: Patient Portal: Self-Sovereign Health Record Access (Part 1)

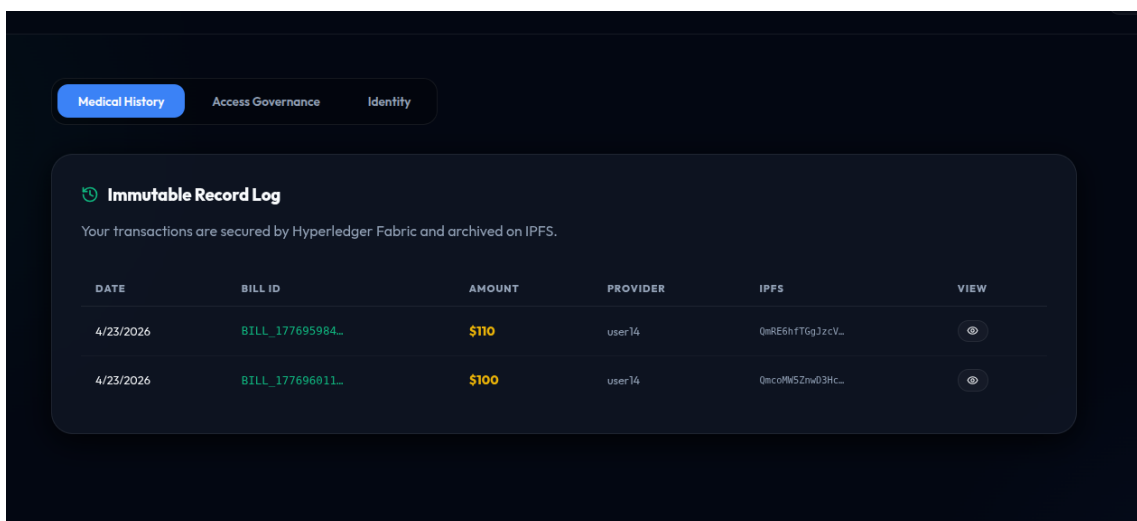


Figure 4.7: Patient Portal: Self-Sovereign Health Record Access (Part 2)

4.2.7 Pharmacist / Billing Portal

The Pharmacist Portal demonstrates **Attribute-Based Access Control (ABAC)** in action. The chaincode verifies the caller’s role and returns only prescription-relevant data (medicine names and quantities), explicitly filtering out private primary diagnoses.

Algorithm 3 Billing Transaction with IPFS Anchoring

```

1: procedure PROCESSBILLING(patientId, medicineList)
2:   Validate patient exists on ledger
3:   doc  $\leftarrow$  Build prescription document (medicines, doses, timestamps)
4:   ipfsHash  $\leftarrow$  UPLOADTOIPFS(doc) ▷ Off-chain storage
5:   billId  $\leftarrow$  Generate unique bill identifier
6:   INVOKECHAINCODE("createBill", billId, patientId, ipfsHash)
7:   for each medicine in medicineList do
8:     if medicine.itemId exists then
9:       INVOKECHAINCODE("deductInventoryItem", medicine.itemId,
10:        quantity)
11:     end if
12:   end for
13:   return success with billId and ipfsHash
14: end procedure

```

Key File: app/billing/src/App.jsx

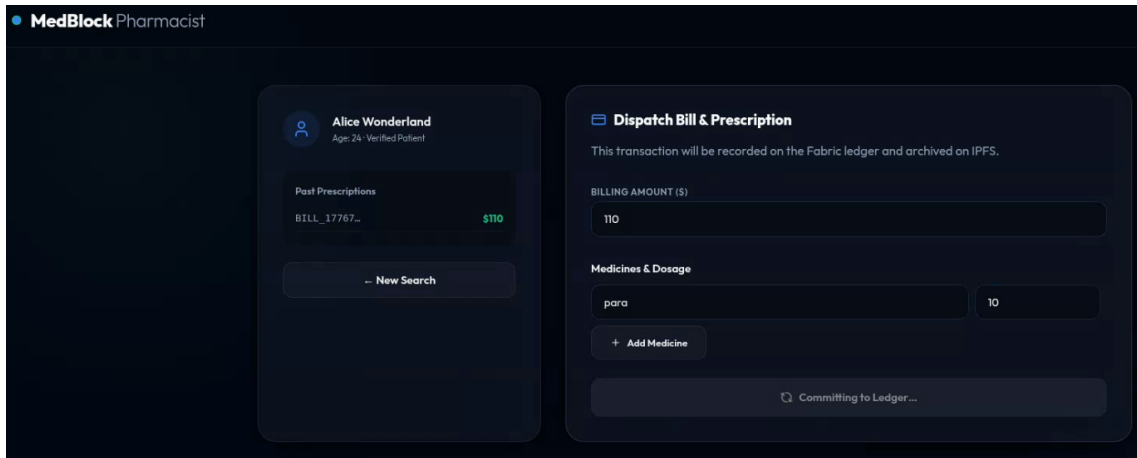


Figure 4.8: Pharmacist Portal: Prescription Fulfillment and Billing Interface (Part 1)

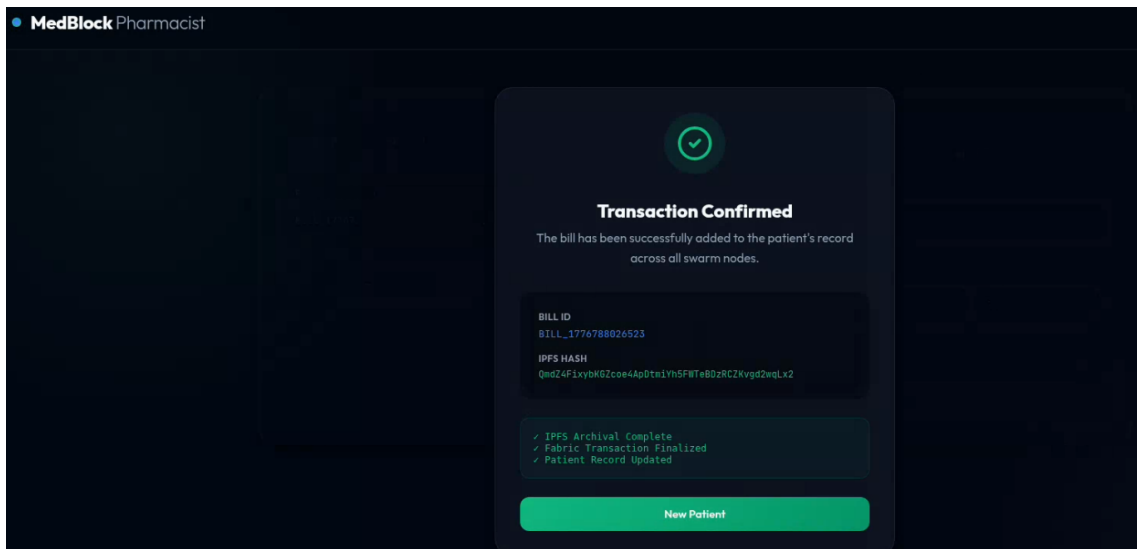


Figure 4.9: Pharmacist Portal: Prescription Fulfillment and Billing Interface (Part 2)

4.3 Ronit singh Rawat (25CSM2S06): API Gateway Architect & Inventory Lead

4.3.1 Role Overview

Ronit is responsible for the Node.js API Gateway that bridges the React frontends to the blockchain via the Fabric SDK, the FileSystemWallet management, and the complete Inventory Application. He operates PC3, ensuring high availability and fault tolerance.

4.3.2 Containers Hosted on PC3

Table 4.4: PC3 Container Inventory (Ronitsingh Rawat)

Container	Image	Purpose
Peer2	hyperledger/fabric-peer:2.5	Redundancy peer for fault tolerance
CouchDB2	couchdb:3.3.2	Mirrored state database
API Gateway	Node.js (Express)	Fabric SDK bridge (REST → gRPC)
Inventory Portal	React (Vite)	Medicine supply chain tracking

4.3.3 API Gateway Architecture

The API Gateway (`app/backend/app.js`) is the central nervous system of the Electronic Health Record (EHR) System. It translates standard HTTP/REST requests from the React frontends into cryptographically signed gRPC transactions using the `fabric-network` SDK.

Core Dependencies:

- `fabric-network` — Official Hyperledger Fabric SDK for Node.js
- `express` — HTTP REST framework
- `axios` — IPFS API communication

- **crypto** — Deterministic identity hashing

4.3.4 Smart Routing Algorithm

A unique feature of the API Gateway is its intelligent transaction routing. Instead of sending all requests to a single peer, the gateway distributes load across the Swarm:

Algorithm 4 Smart Transaction Routing

```

1: procedure SMARTROUTE(functionName, endorsers)
2:   module ← CLASSIFYFUNCTION(functionName)
3:   if module = “billing” and |endorsers| > 1 then
4:     targetPeer ← endorsers[1]                                ▷ Route to Peer1 (PC2)
5:   else if module = “inventory” and |endorsers| > 2 then
6:     targetPeer ← endorsers[2]                                ▷ Route to Peer2 (PC3)
7:   else
8:     targetPeer ← endorsers[0]                                ▷ Default to Peer0 (PC1)
9:   end if
10:  return targetPeer
11: end procedure

```

4.3.5 Wallet and Identity Management

The API manages a **FileSystemWallet** containing X.509 certificates for all registered users. When a frontend request arrives, the gateway:

1. Extracts the user ID from the request body.
2. Hashes the ID deterministically to map it to one of 50 pre-enrolled certificates (**user1** through **user50**).
3. Loads the corresponding private key from the wallet.
4. Creates a Gateway connection using the loaded identity.
5. Signs and submits the transaction to the target peer.

```

1 function mintUserCertificate(id) {
2     if (!id) return 'user1';
3     if (id === 'admin' || id === 'manager') return 'admin';
4     const hash = crypto.createHash('sha256').update(id).
        digest('hex');
5     const index = (parseInt(hash.slice(0, 8), 16) % 50) + 1;
6     return `user${index}`;
7 }

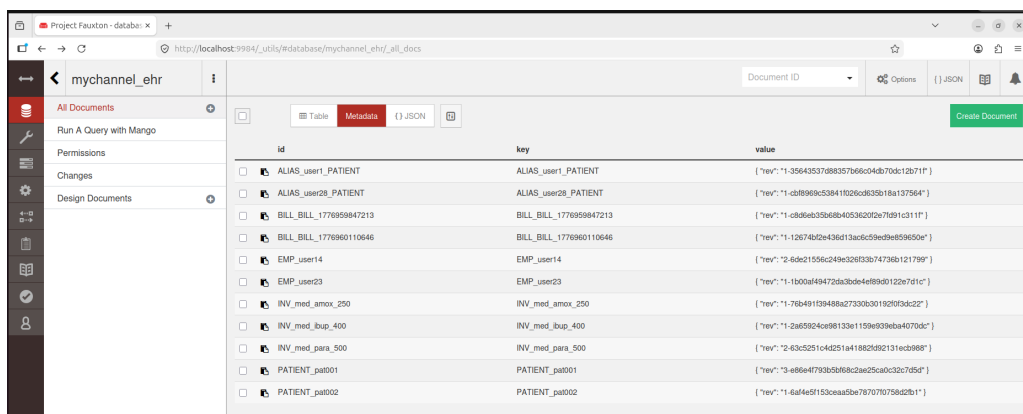
```

Listing 4.4: Identity Mapping Function (app.js)

4.3.6 High Availability and Peer Redundancy

PC3 hosts Peer2, which serves as a redundancy node. Its importance lies in:

- **Fault Tolerance:** If Mohit's PC2 goes offline, the API Gateway reroutes transactions to Peer2, ensuring zero downtime.
- **State Mirroring:** CouchDB2 maintains an exact replica of the world state through the Gossip protocol.
- **Independent Validation:** Peer2 independently validates all blocks received from the Orderer, ensuring no single peer can forge false transactions.



The screenshot shows the CouchDB2 interface for a database named 'mychannel_ehr'. The interface includes a sidebar with navigation options like 'All Documents', 'Run A Query with Mango', 'Permissions', 'Changes', and 'Design Documents'. The main area displays a table with columns 'id', 'key', and 'value'. The table contains several documents, each with a unique key and a corresponding value (represented as a hex string). A 'Create Document' button is visible in the top right corner.

id	key	value
ALIAS_user1_PATIENT	ALIAS_user1_PATIENT	{ "rev": "1-3564353708835766604b70dc12b71f" }
ALIAS_user26_PATIENT	ALIAS_user26_PATIENT	{ "rev": "1-c88969c538411026cd835b18a137564" }
BILL_BILL_1770959847213	BILL_BILL_1770959847213	{ "rev": "1-c88969c538411026cd835b18a137564" }
BILL_BILL_1770960110646	BILL_BILL_1770960110646	{ "rev": "1-1267420e436d13ac6c59ed9e85950e" }
EMP_user14	EMP_user14	{ "rev": "2-8dc21556c249c326f33b74736b121799f" }
EMP_user23	EMP_user23	{ "rev": "1-1b00a49473da3bde4e89d0122e7d1c" }
INV_med_amox_250	INV_med_amox_250	{ "rev": "1-76b49119488a27330c301920f3dc22" }
INV_med_ibup_400	INV_med_ibup_400	{ "rev": "1-2a65924c98133e1159a9398ba4070dc" }
INV_med_para_500	INV_med_para_500	{ "rev": "2-43c5251c44251a11829292131ecb988" }
PATIENT_pat001	PATIENT_pat001	{ "rev": "3-e86e479305b68c2ac25ca0c32c7d5d" }
PATIENT_pat002	PATIENT_pat002	{ "rev": "1-6af4e51153caas5b767070758d2b1" }

Figure 4.10: State Mirroring: CouchDB2 Interface on PC3

4.3.7 Inventory Application

The Inventory Application extends the EHR platform into the hospital supply chain. It tracks medicine stock as digital assets on the Fabric ledger.

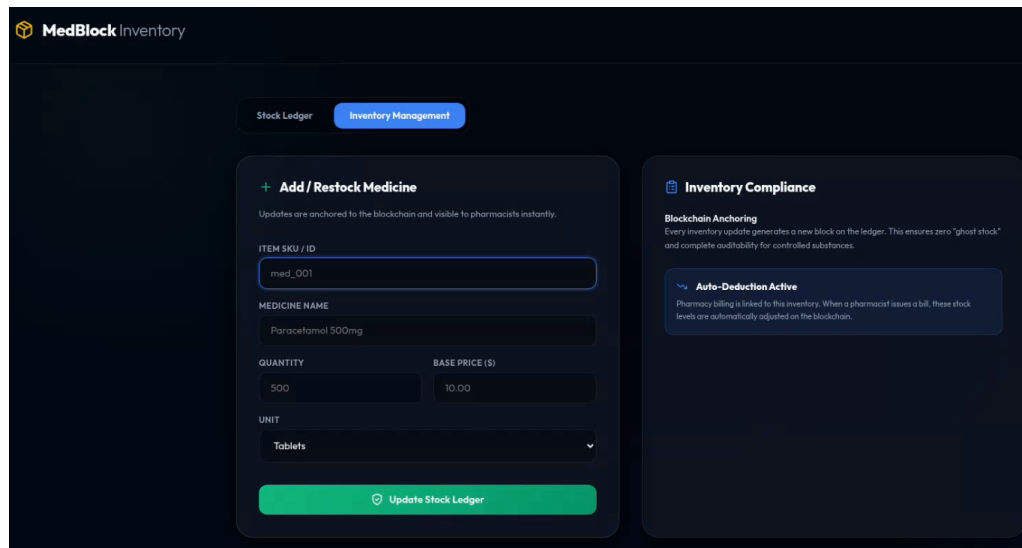
Algorithm 5 Inventory Deduction with Fraud Prevention

```

1: procedure DEDUCTINVENTORY(itemId, quantity)
2:   role  $\leftarrow$  GETROLE(ctx)
3:   if role  $\notin$  {“manager”, “billing”, “inventory”} then
4:     throw “Access Denied”
5:   end if
6:   item  $\leftarrow$  GETSTATE(“INV_” + itemId)
7:   if item = null then
8:     throw “Item not found”
9:   end if
10:  if item.quantity < quantity then
11:    throw “Insufficient stock”
12:  end if
13:  item.quantity  $\leftarrow$  item.quantity – quantity
14:  item.lastDeducted  $\leftarrow$  current timestamp
15:  PUTSTATE(“INV_” + itemId, item)
16:  return item
17: end procedure

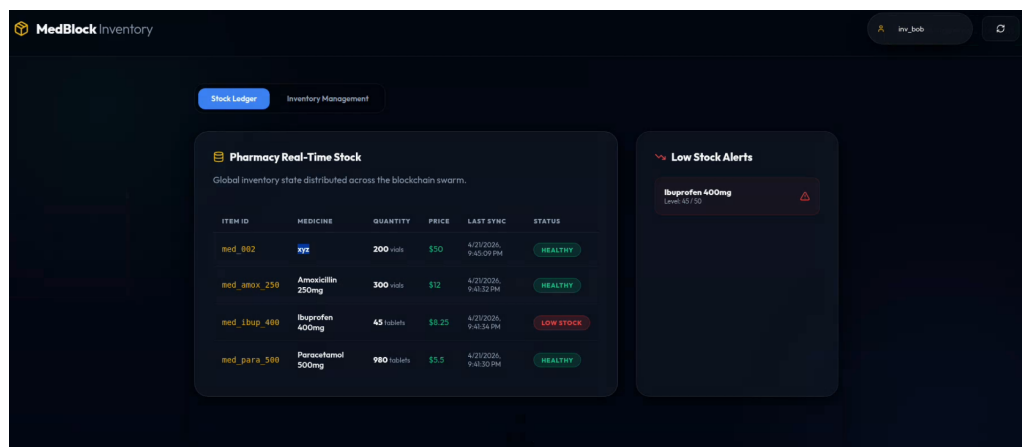
```

Key Feature — Atomic Billing-Inventory Linkage: When a pharmacist fulfills a prescription through the Billing Portal, the API Gateway atomically triggers an `invokeChaincode(‘deductInventoryItem’)` call. Because the blockchain is immutable, it is mathematically impossible for medicine to be dispensed without the ledger permanently recording the stock deduction.



The screenshot shows the 'MedBlock Inventory' application with the 'Inventory Management' tab selected. The main section is titled '+ Add / Restock Medicine' and includes a note: 'Updates are anchored to the blockchain and visible to pharmacists instantly.' Below this is a form with the following fields: 'ITEM SKU / ID' (containing 'med_001'), 'MEDICINE NAME' (containing 'Paracetamol 500mg'), 'QUANTITY' (containing '500'), 'BASE PRICE (\$)' (containing '10.00'), and 'UNIT' (a dropdown menu showing 'Tablets'). A green button at the bottom says 'Update Stock Ledger'. To the right, there is an 'Inventory Compliance' section with two sub-sections: 'Blockchain Anchoring' (explaining that every update generates a new block) and 'Auto-Deduction Active' (explaining that pharmacy billing is linked to the inventory).

Figure 4.11: Inventory Portal: Adding New Medicine Stock to the Ledger



The screenshot shows the 'MedBlock Inventory' application with the 'Stock Ledger' tab selected. The main section is titled 'Pharmacy Real-Time Stock' and includes a note: 'Global inventory state distributed across the blockchain swarm.' Below this is a table with the following columns: 'ITEM ID', 'MEDICINE', 'QUANTITY', 'PRICE', 'LAST SYNC', and 'STATUS'. The table contains four rows of data. To the right, there is a 'Low Stock Alerts' section with a warning icon and a list of items with low stock levels.

ITEM ID	MEDICINE	QUANTITY	PRICE	LAST SYNC	STATUS
med_002	xyz	200 vials	\$50	4/21/2026, 9:35:09 PM	HEALTHY
med_003_250	Amoxicillin 250mg	300 vials	\$12	4/21/2026, 9:41:02 PM	HEALTHY
med_100p_400	Ibuprofen 400mg	45 tablets	\$8.25	4/21/2026, 9:41:34 PM	LOW STOCK
med_para_500	Paracetamol 500mg	980 tablets	\$5.5	4/21/2026, 9:41:50 PM	HEALTHY

The 'Low Stock Alerts' section shows a warning icon and a list of items with low stock levels: 'Ibuprofen 400mg' with a level of 45 / 50.

Figure 4.12: Inventory Portal: Medicine Stock Management and Deduction View

4.4 Verification and Results

4.4.1 Ledger Synchronization Verification

To verify that the Gossip protocol successfully synchronizes data across all three nodes, the following diagnostic commands were executed on each PC:

```
1 # On PC1 (Ankit):
2 docker exec $(docker ps -q -f name=peer0) \
3     peer channel getinfo -c mychannel
4
5 # On PC2 (Mohit):
6 docker exec $(docker ps -q -f name=peer1) \
7     peer channel getinfo -c mychannel
8
9 # On PC3 (Ronit):
10 docker exec $(docker ps -q -f name=peer2) \
11     peer channel getinfo -c mychannel
```

Listing 4.5: Block Height Verification on All Peers

All three peers reported identical block heights and matching `currentBlockHash` values, confirming perfect ledger synchronization across the distributed Swarm network.

4.4.2 CouchDB State Verification

Direct CouchDB queries on each node confirmed that the world state was replicated correctly:

```
1 # On PC2 (Mohit):
2 docker exec $(docker ps -q -f name=couchdb1) \
3     curl -s -u admin:adminpw http://localhost:5984/
4     mychannel_ehr/_all_docs
5
6 # On PC3 (Ronit):
7 docker exec $(docker ps -q -f name=couchdb2) \
```

```
7 curl -s -u admin:adminpw http://localhost:5984/
  mychannel_ehr/_all_docs
```

Listing 4.6: CouchDB State Query on Worker Nodes

4.4.3 System Specifications

Table 4.5: Development and Deployment Environment

Component	Specification
Hyperledger Fabric	v2.5 (LTS)
Docker Engine	v24.x
Docker Swarm	Native orchestration
Node.js	v18.x LTS
React	v18.x (Vite build)
CouchDB	v3.3.2
IPFS	Kubo v0.18+
VPN	Tailscale (WireGuard)
Operating System	Ubuntu 22.04 LTS

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

The Electronic Health Record (EHR) project successfully demonstrates a production-ready, decentralized platform built on Hyperledger Fabric v2.5 and deployed across a three-node Docker Swarm cluster. The project addresses critical shortcomings of centralized healthcare systems by providing:

1. **Data Immutability:** All patient records, prescriptions, and inventory transactions are permanently recorded on the blockchain with cryptographic audit trails, eliminating the possibility of unauthorized data modification.
2. **Patient Self-Sovereignty:** Through ABAC-driven access control and X.509 certificate-based authentication, patients have granular control over who accesses their medical records, including the ability to grant and revoke access to specific healthcare providers.
3. **Decentralized Deployment:** The three-node Swarm architecture connected via Tailscale VPN proves that Hyperledger Fabric can operate across geographically distributed nodes with automatic ledger synchronization via the Gossip protocol.
4. **Supply Chain Integrity:** The atomic linkage between the Billing Portal and Inventory Application ensures that every medicine dispensation is permanently traced on the ledger, preventing pharmaceutical fraud through ghost prescriptions.

5. **Operational Automation:** The single-command `deploy.sh` script automates the entire network lifecycle, from cryptographic key generation to chain-code installation, reducing deployment complexity from hours to minutes.

The team-based distributed development approach, with each member owning specific infrastructure components and application portals, mirrors real-world DevOps practices and demonstrates the practical feasibility of multi-stakeholder blockchain deployments in healthcare settings.

5.2 Future Scope

The following enhancements are planned for future iterations of the Electronic Health Record (EHR) system:

1. **Multi-Organization Support:** Extend the network from a single organization (Org1MSP) to multiple hospital organizations, each with independent Certificate Authorities and private data collections for inter-hospital data sharing.
2. **Byzantine Fault Tolerance:** Replace the Raft Ordering Service with a BFT-based consensus mechanism (such as Hyperledger Fabric's BFT Orderer in v3.0) to tolerate malicious nodes in a multi-organization deployment.
3. **Private Data Collections:** Implement Fabric's Private Data Collections (PDC) to enable selective data sharing between specific organizations without exposing sensitive information to the entire channel.
4. **AI-Powered Analytics:** Integrate machine learning models for predictive medicine inventory management, automated drug interaction detection, and anomaly detection in billing patterns.
5. **Mobile Application:** Develop native mobile applications (Android/iOS) for patient self-service, enabling real-time access to medical records, appointment scheduling, and telemedicine integration.

6. **Zero-Knowledge Proofs:** Incorporate ZKP-based verification to prove medical conditions (e.g., vaccination status) without revealing the underlying patient diagnosis, further strengthening privacy guarantees.
7. **Performance Benchmarking:** Conduct comprehensive throughput and latency benchmarking using Hyperledger Caliper to establish baseline performance metrics under varying transaction loads.

Bibliography

- [1] A. Azaria, A. Ekblaw, T. Vieira and A. Lippman, “MedRec: Using Blockchain for Medical Data Access and Permission Management,” *2016 2nd International Conference on Open and Big Data (OBD)*, pp. 25–30, 2016, doi: 10.1109/OBD.2016.11.
<https://ieeexplore.ieee.org/document/7573685>
- [2] Q. Xia, E. B. Sifah, K. O. Asamoah, J. Gao, X. Du and M. Guizani, “MeD-Share: Trust-Less Medical Data Sharing Among Cloud Service Providers via Blockchain,” in *IEEE Access*, vol. 5, pp. 14757–14767, 2017, doi: 10.1109/ACCESS.2017.2730843.
<https://ieeexplore.ieee.org/document/7990130>
- [3] A. Dubovitskaya, Z. Xu, S. Ryu, M. Schumacher and F. Wang, “Secure and Trustable Electronic Medical Records Sharing using Blockchain,” in *AMIA Annual Symposium Proceedings*, 2017, pp. 650–659.
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5977675/>
- [4] Hyperledger Fabric Documentation, “Key Concepts – Transaction Flow,” Hyperledger Foundation, 2024.
<https://hyperledger-fabric.readthedocs.io/en/release-2.5/txflow.html>
- [5] Hyperledger Fabric Documentation, “Gossip data dissemination protocol,” Hyperledger Foundation, 2024.
<https://hyperledger-fabric.readthedocs.io/en/release-2.5/gossip.html>

- [6] J. Benet, “IPFS - Content Addressed, Versioned, P2P File System,” *arXiv preprint arXiv:1407.3561*, 2014.
<https://arxiv.org/abs/1407.3561>
- [7] Docker Inc., “Swarm mode overview,” Docker Documentation, 2024.
<https://docs.docker.com/engine/swarm/>
- [8] Tailscale Inc., “How Tailscale Works,” Tailscale Documentation, 2024.
<https://tailscale.com/blog/how-tailscale-works>
- [9] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Extended Version),” *USENIX Annual Technical Conference*, pp. 305–319, 2014.
<https://raft.github.io/raft.pdf>
- [10] M. Tanana, “Blockchain and Healthcare: The Estonian Experience,” *2019 IEEE International Conference on Big Data*, pp. 2524–2528, 2019.
<https://ieeexplore.ieee.org/document/9006100>